

Bayesian Machine Learning in Quantitative Finance

Chapter 4: Volatility Surfaces with Sparse Gaussian Processes

Based on Mongwe, Mbuva & Marwala

Chapter 4 Outline

- 1 4.1 Introduction
- 2 4.2 Stochastic Volatility Models
 - 4.2.1 Heston Model
 - 4.2.2 Rough Bergomi Model
- 3 4.3 Model Calibration Using Bayesian Machine Learning
 - 4.3.1 Calibration Framework
 - 4.3.2 Deep Neural Networks and Gaussian Processes
- 4 4.4 Numerical Experiments
 - 4.4.1 Data Description
 - 4.4.2 Experiment Description
- 5 4.5 Results and Discussion
- 6 4.6 Conclusion

4.1 Why Volatility Surfaces? Why Bayesian ML? I

Chapters so far in this series have used Markov chain Monte Carlo (MCMC) and related Bayesian tools to fit models to data. This chapter turns to a concrete, high-stakes application in finance: **fitting stochastic volatility models to the market's option prices**, a task called *calibration*.

The Practical Problem

Every day, an equity index (say the FTSE/JSE All Share Index or the EuroStoxx 50) has hundreds of options trading at different strikes and maturities. A trading desk needs a *single, consistent model* of how the underlying's volatility behaves, so that it can price, hedge, and risk manage options that are *not* directly quoted in the market. Fitting such a model to today's option prices is called **model calibration**.

4.1 Why Volatility Surfaces? Why Bayesian ML? II

Two obstacles making calibration hard:

- **Cost:** realistic stochastic volatility models (Section 4.2) often have no simple closed-form option pricing formula, so pricing a single option can require an expensive Monte Carlo simulation. Calibrating means pricing *many* options, *many* times, while an optimizer searches over parameters – this can be prohibitively slow if done naïvely.
- **Ill-posedness:** calibration is an *inverse problem* – many different parameter combinations can produce nearly the same option prices, so the “best fit” is not unique and small data errors can cause large parameter swings. This calls for *regularization*.

The Machine Learning Fix

Recent literature (Horváth et al., 2021) trains a neural network *offline* to learn the mapping from model parameters to option prices (or implied volatilities). Once trained, this network is a nearly-instant **surrogate** for the expensive pricing step, so calibration to real market data becomes a fast optimization against the surrogate instead of against a slow Monte Carlo pricer.

4.1 Why Volatility Surfaces? Why Bayesian ML? III

Why go further and use Gaussian processes (GPs) instead of (or alongside) neural networks?

- Neural networks are black boxes: hard to interpret, and one must hand-pick an architecture (number of layers, neurons, activation) with no canonical recipe.
- Bayesian methods, including GPs, naturally regularize through a *prior distribution* – helpful for an ill-posed problem.
- GPs additionally report **uncertainty** on every prediction (and hence on every calibrated parameter), which a plain neural network does not.
- Drawback: GPs are much more computationally expensive to train than neural networks (cubic in the number of training points) – motivating the *sparse* GP machinery covered later in this chapter.

This chapter's contribution: a first-in-the-literature Bayesian (Gaussian process) framework for learning volatility surfaces, applied to *both* a developing market (South Africa's FTSE/JSE ALSI) and a developed market (EuroStoxx 50), and compared head-to-head against deep neural networks.

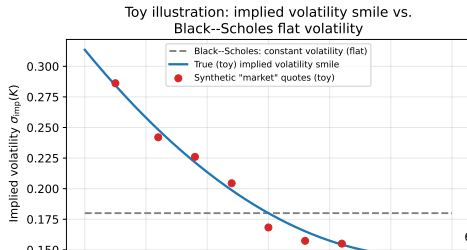
4.1 What Is an Implied Volatility Surface? I

Plain-Language Definition

Take every quoted price of a stock-index option and plug it *backwards* into the Black–Scholes formula, solving for whatever constant volatility number σ would have produced that exact market price. This number is the **implied volatility** for that option's strike K and maturity T . Plotting $\sigma_{\text{imp}}(K, T)$ over the grid of all traded strikes and maturities gives the **implied volatility surface**.

The punchline: if Black–Scholes were literally true, this surface would be perfectly flat – one σ for every strike and maturity. Real markets do not behave this way. Plotted implied volatilities dip and curve, forming a **smile** (U-shape across strike) or a **skew** (a downward slope, common in equity index options because investors pay up for downside/crash protection).

This is direct evidence that the market does *not* believe volatility is constant – it believes volatility itself is random and can jump, which is exactly what a **stochastic volatility model** (Section 4.2) is built to capture.



4.1 What Is an Implied Volatility Surface? II

Reading the toy figure above:

- The dashed grey line is what Black–Scholes assumes: one flat volatility number, regardless of strike.
- The blue curve is a toy (illustrative, not real-market) implied volatility smile: higher for low strikes (deep out-of-the-money puts / crash protection), a minimum somewhere near the at-the-money strike, and a mild upturn for high strikes.
- The red dots are synthetic “market quotes” – noisy observations of the underlying smile, exactly like real bid/ask-derived implied volatilities.

The goal of *calibration* is: given quotes like the red dots (at a handful of strikes and maturities), find the stochastic volatility model parameters that best reproduce this whole curved shape – not just a single flat number.

4.1 Chapter Roadmap

What This Chapter Covers

- 1 **Sec. 4.2** – Two stochastic volatility models: Heston and Rough Bergomi.
- 2 **Sec. 4.3** – The calibration framework, and the two machine learning surrogates: deep neural networks and (sparse) Gaussian processes.
- 3 **Sec. 4.4** – Numerical experiment setup: real option data from two equity markets, and the training/calibration protocol.
- 4 **Sec. 4.5** – Results: speed, parameter stability, fit quality, and the value of GP uncertainty bands.
- 5 **Sec. 4.6** – Conclusion and open questions.

First-in-Literature Claims (per the authors)

- First Bayesian (GP-based) framework for learning volatility surfaces across both developed *and* developing equity markets.
- First use of **sparse** Gaussian processes to calibrate stochastic volatility models to real financial market data.

4.2 What Is a Stochastic Volatility Model? I

Plain-Language Definition

In the Black–Scholes world, volatility σ is a fixed number that never changes. A **stochastic volatility model** instead treats volatility (or variance) as its *own random process*, evolving through time and driven by its own source of randomness – typically correlated with the randomness driving the asset price itself. This lets the model generate exactly the kind of smiles/skews shown in the previous section.

Throughout Section 4.2, we work on a filtered probability space $(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t \in \mathbb{R}}, \mathbb{Q})$ which supports **correlated Brownian motions** – meaning we have two (or more) Brownian motion processes W_t^S (driving the asset price) and W_t^v (driving the variance), whose increments are correlated with each other through a parameter ρ .

4.2 What Is a Stochastic Volatility Model? II

Why bother with two separate models (Heston and Rough Bergomi) in this chapter? They sit at two different points on a trade-off:

- **Heston** (Sec. 4.2.1): a classical, Markovian model. Analytically tractable (closed-form pricing exists via its characteristic function), well understood, but needs 4–5 parameters and does not always fit the very short-maturity skew well.
- **Rough Bergomi** (Sec. 4.2.2): a modern “rough volatility” model. Uses a *fractional* Brownian motion, is *not* Markovian, has no simple closed-form price (Monte Carlo is typically required), but fits a much wider range of volatility surface shapes with just three parameters.

Both models will be calibrated (Sec. 4.4–4.5) to real option data: Heston to the FTSE/JSE ALSI (South Africa), and Rough Bergomi to the EuroStoxx 50 (Europe).

4.2.1 The Heston Stochastic Volatility Model I

The Heston (1993) model extends geometric Brownian motion by letting the *variance* v_t itself follow a random process. It nests Black–Scholes as a special case and remains analytically tractable while capturing the non-log-normal returns, leverage effect, and mean-reverting volatility seen in real markets.

4.2.1 The Heston Stochastic Volatility Model II

Heston Model Dynamics (Eq. 4.1)

$$\begin{aligned}dS_t &= \mu S_t dt + \sqrt{v_t} S_t dW_t^S \\dv_t &= \kappa(\theta - v_t) dt + \xi(t)\sqrt{v_t} dW_t^v \\dW_t^v dW_t^S &= \rho dt\end{aligned}$$

where:

- S_t – the asset (e.g. index) price at time t ;
- μ – the drift term of the asset price;
- v_t – the (instantaneous) **variance** process – this is what makes the model “stochastic volatility”: $\sqrt{v_t}$ plays the role of Black–Scholes’ constant σ , but now it moves randomly through time;
- θ – the long-term average variance that v_t reverts to;
- κ – the rate of mean reversion of v_t toward θ ;
- $\xi(t)$ – the “volatility of volatility”, controlling how wildly v_t itself fluctuates;
- ρ – the instantaneous correlation between the shocks to price and to variance (typically negative for equities: the *leverage effect*, where falling prices coincide with rising volatility).

Python: Simulating the Heston Model

```
import numpy as np

def simulate_heston(S0, v0, mu, kappa, theta, xi, rho,
                    T, n_steps, n_paths, seed=0):
    """Euler discretisation of the Heston (1993) SDE system, Eq. (4.1).
    Returns simulated terminal asset prices S_T (used later to price
    options and back out an implied volatility smile)."""
    rng = np.random.default_rng(seed)
    dt = T / n_steps
    S = np.full(n_paths, S0, dtype=float)
    v = np.full(n_paths, v0, dtype=float)

    for _ in range(n_steps):
        z1 = rng.standard_normal(n_paths)
        z2 = rng.standard_normal(n_paths)
        # correlate the two Brownian shocks via rho (dW^v dW^S = rho dt)
        dW_S = np.sqrt(dt) * z1
        dW_v = np.sqrt(dt) * (rho * z1 + np.sqrt(1 - rho**2) * z2)

        v_pos = np.maximum(v, 0.0) # guard against negative v
        S = S + mu * S * dt + np.sqrt(v_pos) * S * dW_S
        v = v + kappa * (theta - v_pos) * dt + xi * np.sqrt(v_pos) * dW_v

    return S

# Example: FTSE/JSE-ALSI-like parameters (illustrative only)
S_T = simulate_heston(S0=100, v0=0.04, mu=0.0, kappa=2.0, theta=0.04,
```

4.2.2 The Rough Bergomi Stochastic Volatility Model I

Traditional Brownian-motion-based stochastic volatility models (like Heston) struggle to simultaneously produce realistic dynamics *and* the true shape of the volatility surface. **Rough volatility models** address this by substituting a *fractional* Brownian motion for the conventional one in the variance process. The Rough Bergomi model (Bayer, Friz & Gatheral, 2016), an extension of Bergomi (2008), fits a wide variety of volatility surfaces with just three parameters.

4.2.2 The Rough Bergomi Stochastic Volatility Model II

Rough Bergomi Model Dynamics (Eq. 4.2)

$$\begin{aligned}dS_t &= -\frac{1}{2}V_t dt + \sqrt{V_t} dW_t^S \\dV_t &= \xi_0(t) \Xi\left(\sqrt{2H\nu} \int_0^t (t-s)^{H-\frac{1}{2}} dW_s^v\right) \\dW_t^v dW_t^S &= \rho dt\end{aligned}$$

where:

- S_t – the (log) asset price process;
- V_t – the instantaneous variance process, here built from a *stochastic exponential* $\Xi(\cdot)$ applied to a Volterra (fractional) integral, rather than a simple CIR process;
- $H \in (0, 1)$ – the **Hurst parameter**, which controls the *roughness* of the volatility path. Small H (well below 0.5) gives a rough, jagged path – matching the empirically observed roughness of real volatility time series – and controls the decay of the at-the-money forward skew for short maturities;
- $\nu > 0$ – controls the magnitude of volatility fluctuations (an analogue of Heston’s “vol-of-vol” ξ);
- $\xi_0(t) > 0$ – the **initial forward variance curve** (today’s market-implied forward variance term structure), which anchors the model to the current term structure of volatility;

4.2 Heston vs. Rough Bergomi: A Quick Comparison I

	Heston	Rough Bergomi
Driving noise	Standard Brownian motion	Fractional Brownian motion
Free parameters	$\{\mu, \kappa, \theta, \xi, \rho\}$	$\{H, \nu, \rho\}$
Markovian?	Yes (CIR variance)	No (path-dependent)
Closed-form pricing?	Yes (characteristic fn.)	No – Monte Carlo needed
Skew control	ξ (convexity), κ, ρ (slope), θ (level)	H (short-maturity decay) ρ, ν (joint shape)
Used on (this chapter)	FTSE/JSE ALSI (developing)	EuroStoxx 50 (developed)

The common thread: both models require setting a handful of parameters to reproduce an *entire observed volatility surface*. For Heston this can (in principle) be done via semi-analytic pricing formulas; for Rough Bergomi, pricing itself already requires simulation. In both cases, running a full optimizer directly against a Monte Carlo pricer (or even against Heston's more complex closed-form expressions) inside a calibration loop is slow. Section 4.3 introduces the two-step *learn-a-surrogate-then-calibrate* framework used throughout the rest of the chapter to make this fast.

4.3 The Model Calibration Problem I

Definition: Calibration

Given observed market option prices (or, equivalently, an observed implied volatility surface), **calibration** means finding the stochastic volatility model parameters Θ (e.g. $\{\mu, \kappa, \theta, \xi, \rho\}$ for Heston, or $\{H, \nu, \rho\}$ for Rough Bergomi) that make the model's *predicted* implied volatility surface match the *observed* one as closely as possible.

Why is this hard?

- Model calibration is an **ill-posed inverse problem**: many parameter sets can produce very similar-looking surfaces, so the “best fit” is not unique, and small perturbations in the observed data can lead to large, unstable swings in the fitted parameters. This typically requires some form of **regularization** to tame.
- All but the very simplest models require a *numerical* approximation of the option price (e.g. Monte Carlo for Rough Bergomi), so a single evaluation of “how good is this parameter guess?” can itself be expensive – and a calibration routine needs to evaluate this many times as it searches for the best parameters.

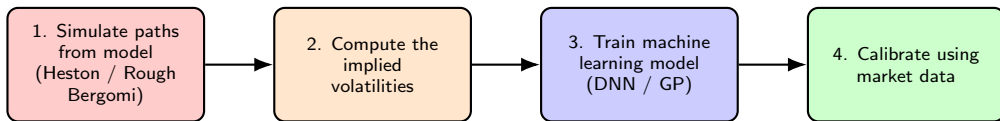
4.3 The Model Calibration Problem II

Why Bayesian Machine Learning?

Bayesian approaches naturally incorporate regularization through the *prior distribution*, which is well suited to an ill-posed problem such as this. This chapter therefore builds a Bayesian framework around Gaussian processes for the calibration task, and compares it against the (non-Bayesian) deep neural network approach already used in the literature.

4.3.1 The Calibration Framework: A Four-Step Pipeline

Rather than optimizing directly against an expensive pricer, this chapter follows a **two-step** calibration procedure (aligned with Horváth et al., 2021): first learn a fast surrogate *offline* using simulated data, then calibrate that surrogate *online* to real market data. (The alternative “one-step”, purely data-driven approach, that skips the formal stochastic volatility model entirely, is left to future work.)



The Four Steps, Explained

- ➊ **Simulate paths:** for a chosen set of model parameters, simulate synthetic asset/variance paths from Heston or Rough Bergomi.
- ➋ **Compute implied volatilities:** from the synthetic data, compute the corresponding option price or implied volatility – the training *label* for that parameter vector. Any numerical pricing method (e.g. Monte Carlo) can be used here.
- ➌ **Train ML model:** learn the mapping (parameters) $\mapsto$ (implied volatility surface) using a machine learning model, *offline*, on the simulated data.
- ➍ **Calibrate to market data:** with the trained (near instant) surrogate in hand, find the parameters

4.3.2 Deep Neural Networks as a Surrogate I

Neural networks are universal function approximators, and this chapter uses a fully connected feed-forward deep network to learn the mapping from model parameters to the (flattened) implied volatility surface.

4.3.2 Deep Neural Networks as a Surrogate II

Feed-Forward Network (Eqs. 4.3–4.4)

$$f_k(x) = b_k + \sum_j v_{jk} h_j(x)$$

$$h_j(x) = \Psi \left(a_j + \sum_i w_{ij} x_i \right)$$

where:

- $x = (x_1, \dots, x_i, \dots)$ – the input vector, here the stochastic volatility model's parameters (e.g. $\{\kappa, \theta, \xi, \rho, v_0\}$ for Heston);
- w_{ij} – the weight connecting the i -th input to the j -th hidden unit;
- a_j, b_k – bias terms for hidden unit j and output unit k ;
- Ψ – the (non-linear) **activation function** that gives the network its ability to approximate non-linear pricing/implied-volatility maps;
- $h_j(x)$ – the activation of the j -th hidden unit;
- v_{jk} – the weight from hidden unit j to output unit k ;
- $f_k(x)$ – the k -th output, here the model implied volatility at the k -th point of the (flattened)

4.3.2 What Is a Gaussian Process? I

Plain-Language Definition

A neural network fits *one* function to the data (one specific curve of best fit). A **Gaussian process (GP)** instead puts a probability distribution directly over *functions*: rather than picking a single best-fit curve, you get a whole *distribution* of plausible curves consistent with the data, together with an uncertainty band around them that is naturally *wide* wherever you have little or no data, and *narrow* near observed data points.

Formally, a Gaussian process is a set of random variables, any finite subset of which is jointly (multivariate) Gaussian distributed. A GP is a **prior over functions**. It is fully specified by two things:

- a **mean function** $\mu(x')$ – our best guess for $f(x')$ before seeing any data (often taken to be zero or a constant);
- a **kernel (covariance) function** $k(x'_i, x'_j)$ – encodes how similar/correlated we believe f at two input points x'_i and x'_j should be (e.g. nearby inputs $\Rightarrow$ similar function values, for a smooth kernel).

4.3.2 What Is a Gaussian Process? II

Why does this matter for calibration? Unlike a deep neural network, once we've fit a GP surrogate for the pricing/IMPLIED-vol map, we get *uncertainty bands* on both the surrogate's predictions and (by propagating through the calibration routine) on the fitted model parameters themselves – valuable information a point-estimate network simply cannot provide.

Cost: training a GP requires inverting an $n \times n$ kernel matrix built from all n training points – an $O(n^3)$ operation. This is the key computational bottleneck that motivates the **sparse** Gaussian process machinery introduced later in this section.

4.3.2 GP Regression: Posterior Mean and Variance I

Given a training dataset $\{\mathbf{x}'_i, \mathbf{x}_t\}_{t=1}^t$ with t *noisy* observations $\mathbf{x}_t = f(\mathbf{x}'_t) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma_X^2 \mathbf{I})$, the GP prior on f implies $f(\mathbf{x}') \sim \mathcal{N}(\mu_{\mathbf{x}'}, \mathbf{K}_{\mathbf{x}'} + \sigma_X^2 \mathbf{I})$.

Notation Note

This chapter's own notation uses $\mathbf{X}$ for the (noisy) observed *outputs* and $\mathbf{x}'$ for the *inputs* – the reverse of many textbooks' convention (which often use X for inputs, y for outputs). We reproduce the book's equations exactly below, then re-derive the same result with the more common x -inputs/ y -outputs notation on the following slide to avoid any confusion.

Posterior Mean and Variance (Eqs. 4.5–4.6)

$$\mathbb{E}(f_*) = \mu_{\mathbf{x}'_*} + k_* (K_{\mathbf{x}'} + \sigma_X^2 \mathbf{I})^{-1} (\mathbf{X} - \mu_{\mathbf{x}'})$$

$$\text{var}(f_*) = k_{**} - k_*^\top (K_{\mathbf{x}'} + \sigma_X^2 \mathbf{I})^{-1} k_*$$

where $k_{**} = k(\mathbf{x}'_*, \mathbf{x}'_*)$ (prior variance at the test point) and $k_* = k(\mathbf{x}', \mathbf{x}'_*)$ (covariances between all training inputs and the test input).

4.3.2 GP Regression: Posterior Mean and Variance II

Hyperparameter Fitting (Eq. 4.7)

$$\log p(\mathbf{X} \mid \mathbf{X}') = \log \mathcal{N}(\mu_{x'}, \mathbf{K}_{x'x'} + \sigma_X^2 \mathbf{I})$$

The kernel hyperparameters (e.g. lengthscale, signal variance) and the noise level σ_X are found by maximizing this **log-marginal likelihood** – i.e. by asking which hyperparameters make the *observed* data most probable under the GP model.

Step-by-Step: Deriving the GP Posterior (Standard Notation) I

Let's rebuild the same result with cleaner, more standard notation: inputs $x_1, \dots, x_n$, noisy targets $y_i = f(x_i) + \epsilon_i$ with $\epsilon_i \sim \mathcal{N}(0, \sigma_n^2)$ i.i.d., and a test input x_* . Assume (for simplicity) a zero mean function.

Step 1 – Joint distribution. Under the GP prior, the training targets $\mathbf{y} = (y_1, \dots, y_n)^\top$ and the (noise-free) function value f_* at the test point are *jointly Gaussian*:

$$\begin{bmatrix} \mathbf{y} \\ f_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} K + \sigma_n^2 I & \mathbf{k}_* \\ \mathbf{k}_*^\top & k_{**} \end{bmatrix}\right)$$

where $K_{ij} = k(x_i, x_j)$, $(\mathbf{k}_*)_i = k(x_i, x_*)$, and $k_{**} = k(x_*, x_*)$.

Step-by-Step: Deriving the GP Posterior (Standard Notation) II

Step 2 – Condition on the observed data. Using the standard formula for conditioning a joint Gaussian on part of its variables gives the predictive distribution $f_* \mid \mathbf{y}$:

$$\mathbb{E}[f_* \mid \mathbf{y}] = \mathbf{k}_*^\top (K + \sigma_n^2 I)^{-1} \mathbf{y}$$

$$\text{Var}[f_* \mid \mathbf{y}] = k_{**} - \mathbf{k}_*^\top (K + \sigma_n^2 I)^{-1} \mathbf{k}_*$$

This is exactly the structure of the book's Eqs. (4.5)–(4.6): the posterior mean is a weighted combination of the observed targets (weights determined by how correlated each training input is with the test input, and de-noised via σ_n^2), and the posterior variance is the *prior* variance k_{**} *shrunk* by an amount that grows with how much relevant training data ($\mathbf{k}_*$) we have nearby.

Key intuition: far from any training data, $\mathbf{k}_* \approx \mathbf{0}$ (nothing correlates with the test point), so the mean reverts to the prior mean (zero here) and the variance reverts to the prior variance k_{**} – i.e. maximal uncertainty. Near training data, the opposite happens: high confidence, narrow uncertainty band.

Worked Numeric Example: GP Posterior by Hand I

Let's compute an exact (small, non-sparse) GP posterior with just **3 training points**, using the **squared exponential kernel** $k(x, x') = \sigma_f^2 \exp(-\frac{1}{2}(x - x')^2/\ell^2)$ with $\sigma_f = 1$, lengthscale $\ell = 1$, and noise $\sigma_n = 0.1$.

Data

$x = (-1, 0, 1)$, $y = (0.9, 0.0, -0.9)$ (roughly $y \approx -x$), test point $x_* = 0.5$.

Step 1: build $K + \sigma_n^2 I$. Using $k(-1, 0) = k(0, 1) = e^{-0.5} \approx 0.6065$ and $k(-1, 1) = e^{-2} \approx 0.1353$:

$$K + \sigma_n^2 I = \begin{pmatrix} 1.0100 & 0.6065 & 0.1353 \\ 0.6065 & 1.0100 & 0.6065 \\ 0.1353 & 0.6065 & 1.0100 \end{pmatrix}$$

(the $1.01 = \sigma_f^2 + \sigma_n^2 = 1 + 0.01$ on the diagonal).

Worked Numeric Example: GP Posterior by Hand II

Step 2: build $\mathbf{k}_*$ and k_{} .**

$$\mathbf{k}_* = (k(-1, 0.5), k(0, 0.5), k(1, 0.5)) = (0.3247, 0.8825, 0.8825), \quad k_{**} = 1.0$$

Step 3: solve the linear system and combine. Solving $(K + \sigma_n^2 I) \boldsymbol{\alpha} = \mathbf{y}$ for $\boldsymbol{\alpha}$ and then computing $\mathbf{k}_*^\top \boldsymbol{\alpha}$ (equivalent to the formulas above) gives:

Result (verified numerically)

$$\mathbb{E}[f_* \mid \mathbf{y}] \approx -0.5740, \quad \text{Var}[f_* \mid \mathbf{y}] \approx 0.0250, \quad \text{std} \approx 0.1582$$

Sanity check: the true underlying trend is $y \approx -x$, so at $x_* = 0.5$ we would guess $y \approx -0.5$; the GP posterior mean -0.574 is close to this but pulled slightly by the noisy training values and the kernel's smoothing, exactly as expected, with a modest posterior uncertainty because $x_* = 0.5$ sits between two training points.

4.3.2 Sparse Gaussian Processes: Why and How I

The Problem

Exact GP training/inference costs $O(n^3)$ in the number of training points n (from inverting the $n \times n$ kernel matrix). This chapter's experiments simulate **40,000** training smiles per model (Sec. 4.4.2) – far too many for an exact GP to handle directly.

Plain-Language Fix: Inducing Points

A **sparse Gaussian process** makes GPs tractable on large datasets by summarizing the data with a small number of well-chosen “**pseudo-points**” (also called **inducing points**), $m \ll n$. Instead of inverting an $n \times n$ matrix, the model only ever needs to invert $m \times m$ matrices, dramatically lowering both the computational and memory cost – at the price of an *approximation*.

- The number of inducing points controls the quality of the approximation: more inducing points $\Rightarrow$ better approximation of the true (full) GP, but higher cost – the user gets to trade off memory/compute against accuracy.

4.3.2 Sparse Gaussian Processes: Why and How II

- Because sparse GPs typically lack a closed-form solution, they are trained via **variational inference**: Bayesian inference reframed as an optimization problem, maximizing a lower bound on the log-marginal likelihood.

4.3.2 Sparse Gaussian Processes: Why and How III

Single-output vs. multi-output sparse GPs (as used in this chapter):

- **Single-output sparse GP:** fit one independent sparse GP *per point* on the volatility surface grid (e.g. 30 separate small GPs for the ALSI grid).
- **Multi-output sparse GP:** fit *one* sparse GP that jointly models the *whole surface at once* as a vector-valued output, allowing the model to implicitly learn connectivity/correlation between different points on the surface. In this chapter, the inducing points and kernel are *shared* across all outputs of the multi-output GP.
- Both variants here use the **squared exponential kernel**.

This chapter's numerical experiments (Sec. 4.4–4.5) compare both sparse GP variants against the deep neural network surrogate of Sec. 4.3.2.

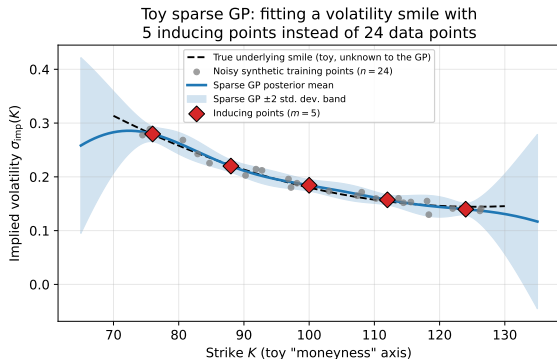
Toy Running Example: Sparse GP Fit to a Synthetic Smile

Our own illustrative simulation (not from the book): 24 noisy synthetic (strike, implied-vol) points sampled around a toy smile, fit with a sparse GP using just **5 inducing points** (squared-exponential kernel, lengthscale ≈ 16).

Numeric summary (posterior mean $\pm$ std):

- near data-rich centre ($K = 100$): mean ≈ 0.184 , std ≈ 0.005 – *narrow* band;
- near the edges ($K = 70, 130$, sparser data): mean $\approx 0.28, 0.13$, std ≈ 0.04 – *wider* band.

This is the hallmark GP behaviour: uncertainty shrinks near data (and near inducing points) and grows away from it.



Python: A Sparse GP (DTC Approximation) From Scratch

```
import numpy as np

def se_kernel(x1, x2, ell, sigma_f):
    """Squared exponential kernel  $k(x, x') = \sigma_f^2 \exp(-0.5(x-x')^2/\ell^2)$ ."""
    d2 = (x1.reshape(-1, 1) - x2.reshape(1, -1)) ** 2
    return sigma_f**2 * np.exp(-0.5 * d2 / ell**2)

def sparse_gp_predict(x_train, y_train, x_inducing, x_star,
                      ell, sigma_f, sigma_n):
    """Sparse GP regression via the Deterministic Training Conditional (DTC)
    approximation: only  $m \times m$  matrices ( $m$  = number of inducing points) are
    ever inverted, instead of the full  $n \times n$  kernel matrix."""
    Kmm = se_kernel(x_inducing, x_inducing, ell, sigma_f) + 1e-6 * np.eye(len(x_inducing))
    Knm = se_kernel(x_train, x_inducing, ell, sigma_f)
    Ksm = se_kernel(x_star, x_inducing, ell, sigma_f)

    Kmm_inv = np.linalg.inv(Kmm)
    Sigma = Kmm + (Knm.T @ Knm) / sigma_n**2          # only  $m \times m$  to invert
    Sigma_inv = np.linalg.inv(Sigma)

    mean = (Ksm @ Sigma_inv @ (Knm.T @ y_train)) / sigma_n**2
    var = (sigma_f**2
           - np.einsum('ij,jk,ik->i', Ksm, Kmm_inv, Ksm)
           + np.einsum('ij,jk,ik->i', Ksm, Sigma_inv, Ksm))
    return mean, np.sqrt(np.clip(var, 1e-10, None))
```

5 inducing points summarising 24 noisy training observations (toy example)

4.4.1 Data Description: Two Equity Markets I

Real option market data from **100 consecutive trading days in 2019** is used for both markets, giving a like-for-like comparison of a developing and a developed market.

FTSE/JSE ALSI Option Data (South Africa – Developing Market)

Implied volatilities on a discrete 2-D grid of moneyness M and time to maturity T :

$$M := \{80\%, 90\%, 100\%, 110\%, 120\%\}, \quad T := \{1M, 2M, 3M, 6M, 9M, 1Y\}$$

giving $5 \times 6 = \mathbf{30}$ points on each implied volatility surface. Calibrated using the **Heston** model.

EuroStoxx 50 Option Data (Europe – Developed Market)

$$M := \{50\%, 60\%, \dots, 150\%\} \text{ (11 values, step 10\%)}, \quad T := \{0.1Y, 0.3Y, 0.6Y, 0.9Y, 1.2Y, 1.5Y, 1.8Y, 2.0Y\}$$

giving $11 \times 8 = \mathbf{88}$ points on each implied volatility surface. Calibrated using the **Rough Bergomi** model.

4.4.1 Data Description: Two Equity Markets II

Why pair Heston with ALSI and Rough Bergomi with EuroStoxx? This mirrors the chapter's aim of demonstrating the Bayesian (sparse GP) calibration framework across *both* a classical, tractable model on a developing market, and a modern rough-volatility model on a developed market – rather than an exhaustive cross-product of every model/market combination.

Note also the very different *grid sizes*: 30 points for ALSI vs. 88 for EuroStoxx. This directly affects the machine learning surrogates' output dimensionality (Sec. 4.3.2, 4.4.2) – e.g. the neural network's number of output nodes must match the grid size for each market.

4.4.2 Experiment Description: Training and Calibration Setup I

Offline Training Data (Simulated)

For each stochastic volatility model, **40,000 training examples** were simulated: 40,000 Heston smiles, and (separately) 40,000 Rough Bergomi smiles. Each example pairs a parameter vector with its resulting implied volatility surface (computed numerically, e.g. via Monte Carlo where needed).

Machine Learning Models Compared

- **Deep neural network (DNN):** 3 hidden layers of 30 neurons each, **ELU** activation, linear output layer; number of output nodes = surface size (30 for ALSI, 88 for EuroStoxx).
- **Single-output sparse GP:** one sparse GP per surface point, squared exponential kernel, **100 inducing points** each.
- **Multi-output sparse GP:** one joint sparse GP over the whole surface, squared exponential kernel, **1,000 inducing points**.

4.4.2 Experiment Description: Training and Calibration Setup II

Online Calibration and Evaluation

All three trained models are then calibrated *online* to the real FTSE/JSE ALSI and EuroStoxx 50 market data (100 trading days each), using a standard calibration routine against the (now near-instant) surrogate. Performance is assessed using the **root mean square error (RMSE)** across the *entire* volatility surface, together with the computational resources required by each method.

All results in the book were produced in Python on a 64-bit machine.

Python: Training-Data Simulation & Surrogate Pipeline

```
import numpy as np

def build_training_set(n_examples, simulate_fn, price_to_impliedvol_fn,
                      strikes, maturities, seed=0):
    """Steps 1-2 of the calibration framework: simulate model paths for
    randomly sampled parameter vectors, then convert to an implied
    volatility surface (the training label)."""
    rng = np.random.default_rng(seed)
    X_params, Y_surfaces = [], []
    for _ in range(n_examples):
        theta = sample_parameter_vector(rng) # e.g. Heston/RoughBergomi params
        paths = simulate_fn(theta, strikes, maturities)
        iv_surface = price_to_impliedvol_fn(paths, strikes, maturities)
        X_params.append(theta)
        Y_surfaces.append(iv_surface.flatten())
    return np.array(X_params), np.array(Y_surfaces)

# Step 3: train a surrogate offline (DNN shown; GP analogous, see prior slide)
X_train, Y_train = build_training_set(40_000, simulate_heston,
                                     price_to_impliedvol, strikes, maturities)

surrogate = train_dnn(X_train, Y_train, hidden=(30, 30, 30), activation='elu')

# Step 4: calibrate to real market data using the fast surrogate
from scipy.optimize import minimize
market_surface = load_market_iv_surface('ALSI') # 30-point grid
loss = lambda theta: np.sqrt(np.mean((surrogate.predict(theta) - market_surface) ** 2))
result = minimize(loss, x0=initial_guess, method='Nelder-Mead')
```

4.5 Speed and Parameter Stability I

Computational Speed

As expected, the **deep neural network was the fastest** method. The **single-output sparse GP was the slowest**, at around **6 times slower** than the DNN. This highlights the main practical drawback of the GP approach: even with sparsity, GP training/evaluation remains more expensive than a neural network, motivating future work on distributed/scalable GP methods.

Parameter Stability Over Time (Heston on FTSE/JSE ALSI)

All three methods (multi-output sparse GP, DNN, single-output sparse GP) produce *similar* Heston parameter estimates across the 100 trading days for most parameters. The **deep neural network showed the most instability** in its parameter estimates through time, particularly for ν and θ – i.e. the DNN's calibrated parameters jumped around more from day to day than the GP-based methods' did.

4.5 Speed and Parameter Stability II

Why might the GP-based methods be more stable? Because GPs are Bayesian, their predictions (and hence their implied calibration loss surface) are naturally smoothed/regularized relative to a plain point-estimate neural network – one of the practical benefits motivating this chapter's Bayesian framework, beyond simply reporting uncertainty bands.

4.5 Fit Quality: Best- and Worst-Calibration Days I

On the days with the largest calibration error (for each market): in general all three methods produce very similar fits to the data, *except* for the multi-output sparse GP, whose fit visibly deviates from the other two as maturity increases. The two GP approaches show similar confidence bands on the ALSI dataset, though the single-output GP produces noticeably *narrower* (lower) one-standard-deviation bands than the multi-output GP.

For the **smallest maturity** on the EuroStoxx 50 data, *all three* methods fail to fully capture the skew around the at-the-money point on the worst day – this particular skew was unusually steep compared to the market's typical skew over the study period.

4.5 Fit Quality: Best- and Worst-Calibration Days II

Across both datasets, a consistent pattern emerges in the **confidence bands**: the *lowest* maturity has the *narrowest* confidence bands overall, with bands widening as maturity increases; and within a given maturity, bands are narrowest around the at-the-money moneyness level, widening as one moves away from at-the-money.

On the days with the smallest calibration error, all three methods fit the data noticeably better – in particular, all three fully capture the EuroStoxx 50 skew at the smallest maturity on these “easy” days (in contrast to the worst-day case above). The multi-output GP’s fit still deviates from the other two as maturity increases, and the same pattern of narrower bands at low maturity / wider bands at high maturity persists.

4.5 RMSE Through Time and Overall Ranking I

RMSE Across the Whole Surface, Over Time

Tracking the root mean square error across the entire volatility surface through all 100 trading days shows that the calibration error is fairly stable for all three methods, though there are periods with noticeably larger errors – possibly indicating that a larger training set, or a different/better-specified stochastic volatility model, would be needed on those particular days.

Which Method Performs Worst Overall?

On *both* datasets, the **multi-output sparse GP is the poorest performer**, with a consistently higher RMSE than the other two methods through time. This is explained by its comparatively poor fit at **large maturities** on the volatility surface, as noted in the smile-by-smile comparisons above. On the FTSE/JSE ALSI data specifically, the DNN and single-output GP track each other closely; on EuroStoxx 50, the single-output GP performs similarly to the DNN, with the multi-output GP trailing well behind both.

4.5 RMSE Through Time and Overall Ranking II

The Headline Result

The (Bayesian, sparse GP) probabilistic approach achieves **similar predictive performance to the deep neural network** – the current state-of-the-art – while additionally providing calibrated **uncertainty estimates** on both the fitted volatility surface and (by extension) the calibrated model parameters. This is valuable to practitioners who need to understand not just their best single estimate of the smile shape, but how confident they should be in it across different maturities.

4.5 Toy Illustration vs. the Book's Real-Market Results

Keep These Two Firmly Separate

- **Our toy example** (earlier in Sec. 4.3.2): a synthetic, hand-constructed smile with 24 noisy points and **5** inducing points – built purely to give visual and numerical intuition for how a sparse GP posterior behaves.
- **The book's actual experiment** (this section): real FTSE/JSE ALSI and EuroStoxx 50 market data over **100 trading days**, with **40,000** simulated training smiles per stochastic volatility model, and **100** (single-output) or **1,000** (multi-output) inducing points.

The scales are wildly different, but the *mechanism* is identical: a handful of well-placed inducing points summarizing a much larger dataset, producing a posterior mean and an uncertainty band that widens where data (or inducing points) are sparse – exactly what our toy figure shows on a small scale, and what the book's real-market results show at production scale.

4.6 Conclusion I

This chapter used **sparse Gaussian processes** to calibrate the Heston and Rough Bergomi stochastic volatility models to real option volatility data across a developing market (FTSE/JSE ALSI) and a developed market (EuroStoxx 50). Both single-output (per-surface-point) and multi-output (whole-surface-at-once) sparse GPs were considered, and benchmarked against the current state-of-the-art deep neural network approach.

Key Findings

- The (Bayesian) probabilistic approach produces results **similar** to the deep neural network, with the added advantage of **capturing uncertainty** in the calibration.
- On the **FTSE/JSE ALSI** data, the single- and multi-output GPs produce similar results, with narrow confidence bands at lower moneyness/maturity and wider bands as moneyness and maturity increase.
- On the **EuroStoxx 50** data, the multi-output GP is the poorest performer, with visibly wide confidence bands reflecting this; the single-output GP produces results similar to the neural network, with narrower confidence bands than the multi-output GP.

4.6 Conclusion II

Limitations and Future Work

- **Shared inducing points/kernel:** the multi-output GP's weaker EuroStoxx 50 performance may be improved by *not* sharing inducing points and kernels across the surface's output dimensions.
- **No explicit no-arbitrage constraints:** like the neural network literature this chapter compares against, the approximated pricing function is not guaranteed to be arbitrage-free. Incorporating no-arbitrage conditions explicitly is left to future work.
- **Computational cost:** the main drawback of the GP approach versus neural networks remains its higher computational complexity (recall: single-output GP $\approx 6\times$ slower than the DNN). Future work will explore **distributed/scalable GP** approaches (e.g. product-of-experts methods) to address this.

4.6 Looking Ahead

Where This Chapter Sits in the Bigger Picture

Model calibration here relied on a *formal* stochastic volatility model (Heston or Rough Bergomi) plus a machine learning surrogate to speed up fitting it to data – and, as just discussed, this approach does not automatically guarantee arbitrage-free prices.

Next Chapter Preview

The next chapter takes a different route: a **normalizing flows**-based approach to pricing options in an emerging market. By learning the **risk-neutral density** directly, this approach does not need to impose no-arbitrage conditions as an extra constraint on a loss function – the learned density already captures them by construction.